

AutoJudge: Problem Difficulty Classification and Scoring System

Vodnala Rishik ,24116107

B. Tech Electronics and Communication Engineering
Indian Institute of Technology Roorkee

January 8, 2026

Contents

1	Introduction	2
2	Dataset Description	2
3	Data Preprocessing	2
4	Feature Engineering	3
5	Model Architecture	3
5.1	Classification Model	3
5.2	Regression Model	3
6	Experimental Setup	3
7	Results and Evaluation	3
7.1	Classification Performance	3
7.2	Regression Performance	4
8	Web Interface	4
9	Conclusion and Future Work	4

1 Introduction

Competitive programming platforms host thousands of coding problems with varying levels of difficulty. Correctly identifying the difficulty level of a problem is important for learners to select appropriate challenges and for platforms to organize problem sets effectively.

This project proposes **AutoJudge**, a machine learning-based system that automatically predicts the difficulty level (Easy, Medium, or Hard) and a numerical difficulty score (1–10) for coding problems using their textual descriptions.

The system uses Natural Language Processing (NLP) techniques combined with classification and regression models. A web-based interface allows users to obtain instant predictions for unseen problems.

2 Dataset Description

A custom dataset consisting of competitive programming problems was used for this project. Each problem entry contains:

- Problem title
- Problem description
- Input specification
- Output specification

Each problem is labeled with:

- A categorical difficulty label: Easy, Medium, or Hard
- A numerical difficulty score on a scale of 1 to 10

The dataset was split into training and testing subsets to evaluate the performance of the models on unseen data.

3 Data Preprocessing

Textual preprocessing was applied to ensure consistency and reduce noise. The following steps were performed:

- Conversion of text to lowercase
- Removal of unnecessary whitespace
- Concatenation of problem title, description, input, and output sections

The preprocessed text was used as input for feature extraction.

4 Feature Engineering

To convert textual data into numerical form, the TF-IDF (Term Frequency–Inverse Document Frequency) vectorization technique was employed.

TF-IDF captures both the importance of a word within a document and its relevance across the entire corpus, making it suitable for text classification tasks.

5 Model Architecture

5.1 Classification Model

A supervised machine learning classifier was trained to categorize problems into Easy, Medium, or Hard difficulty levels based on TF-IDF features.

5.2 Regression Model

A regression model was trained using the same feature representation to predict a continuous difficulty score between 1 and 10.

Both models were trained on the same dataset and saved as serialized files for reuse during inference.

6 Experimental Setup

The dataset was split into training and testing sets using an 80:20 ratio. The models were trained on the training set, while the test set was reserved for performance evaluation.

The experiments were conducted using Python and the scikit-learn library.

7 Results and Evaluation

7.1 Classification Performance

The classification model achieved an accuracy of **56.62%** on the test dataset.

Table 1 shows the confusion matrix, which provides a detailed breakdown of correct and incorrect predictions across difficulty classes.

Table 1: Confusion Matrix for Difficulty Classification

Actual \ Predicted	Easy	Medium	Hard
Easy	48	25	63
Medium	21	53	188
Hard	23	37	365

The results indicate that most misclassifications occur between Medium and Hard problems, which is expected due to overlapping complexity levels.

7.2 Regression Performance

The regression model achieved the following performance:

- Mean Absolute Error (MAE): **1.723**
- Root Mean Squared Error (RMSE): **2.054**

These results demonstrate that the model is able to estimate difficulty scores with reasonable accuracy.

8 Web Interface

A web-based user interface was developed using Streamlit. The interface allows users to input a coding problem description and receive instant predictions.

Figure 1 shows a screenshot of the web interface during execution.

The screenshot displays the AutoJudge web interface. At the top, there is a text input field containing the problem description: "plane. For each planning scenario, determine whether the first cooperative can pick two distinct pumps and the second cooperative can pick two distinct reservoirs so that the two straight pipes intersect. If this is possible, report the indices of those pumps and reservoirs; otherwise declare that the project cannot be realized." To the right of the input field, the text "2-105" is visible. Below the input field is a red button labeled "Predict Difficulty". To the right of the input field, there is a section titled "Output Description" with the text: "In case several valid solutions exist, any one of them is acceptable." Below the "Predict Difficulty" button, the "Prediction Results" section is shown. It features the text "Predicted Class: hard" in red, followed by "Difficulty Score (1-10 Scale)" and the score "5.94".

Figure 1: AutoJudge Web Interface

9 Conclusion and Future Work

This project demonstrates the feasibility of automatically predicting problem difficulty using machine learning and NLP techniques.

Future improvements may include:

- Incorporating code-level features
- Training on larger and more diverse datasets
- Improving regression accuracy with advanced models